الجـــمهوريـــة الجزائريـــة الديمـــقراطيـــــة الشـعبيـة
ⵜⴰⵣⴰⵡⴰⵜ ⵜⴰⵎⴰⵔⴰⵖⵉⵜ ⵜⴰⵣⴰⵢⴰⵔⵜ ⵜⴰⵡⵙⵙⵔⵜ
République Algérienne Démocratique et Populaire
وزارة التعليـــم العـــالي والبـــحث العلمـــي
ⴰⵢⵏⴰⵙ ⵏ ⵜⵓⵙⵎⵙⴰ ⵜⵉⵣⵣ ⴰ ⵜⴰⵏⴰⵙ ⵜⵓⵓⵏⴰⵏ
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

ECOLE NATIONALE SUPÉRIEURE D'INFORMATIQUE

المدرسة الوطنية العليا للإعلام الآلي
ⴰⵢⵜⵓⵎⴰⵡⵉ ⴰⵢⵏⵎⵓⵍ ⵜⵉⵣⵣ ⵏ ⵜⵓⵜⵍⵓⵎⴰⵔⵉⵜ
École nationale Supérieure d'Informatique

CPI 1 **Année 2021 / 2022**

TP2
EN ALGORITHMIQUE ET STRUCTURES DE DONNEES STATIQUES (ALSDS)

REALISE PAR :
ARROUCHE Abdellah
MENASSEL Rayane Ibrahim

Section : D

Groupe N° : 11

Semestre 1

Table de Matières

I. Introduction 2
II. Sujet du TP2 2
III. Solution Proposé 3
1. Découpage modulaire et sa justification 3
2. Conception et réalisation 4
1-CONCTPOS 4
2-Cipher 6
3-Algorithme principal 8
IV. Conclusion 13

I. Introduction

La confidentialité étant très importante dans les communications secrètes, l'être-humain a pensé à protéger ses documents ; C'est la stéganographie ! (Du grec : steganós « étanche » et graphḗ « écriture »).

Dans « L'Enquête », Hérodote a cité un des plus connus exemples de la stéganographie :

> « Il fit raser la tête de son esclave le plus fidèle, lui tatoua son message sur le crâne et attendit que les cheveux eussent repoussé ; quand la chevelure fut redevenue normale, il fit partir l'esclave pour Milet. » (Livre V, 35)

Mais Jules César a opté pour une méthode plus rigoureuse qui consiste à substituer les lettres de l'alphabète ; C'est un chiffrement, d'où vient la cryptographie, une notion intimement liée au technologies et machines : les mots de passe, le hash (la fonction de hachage), protection des informations, …

Basé sur le code de César, le chiffre de Vigenère est un chiffrement tellement complexe, il n'était pas déchiffré pour 310 ans. On traitera ce « chiffre indéchiffrable » ci-dessous, mais d'une manière simplifiée comme ce suit :

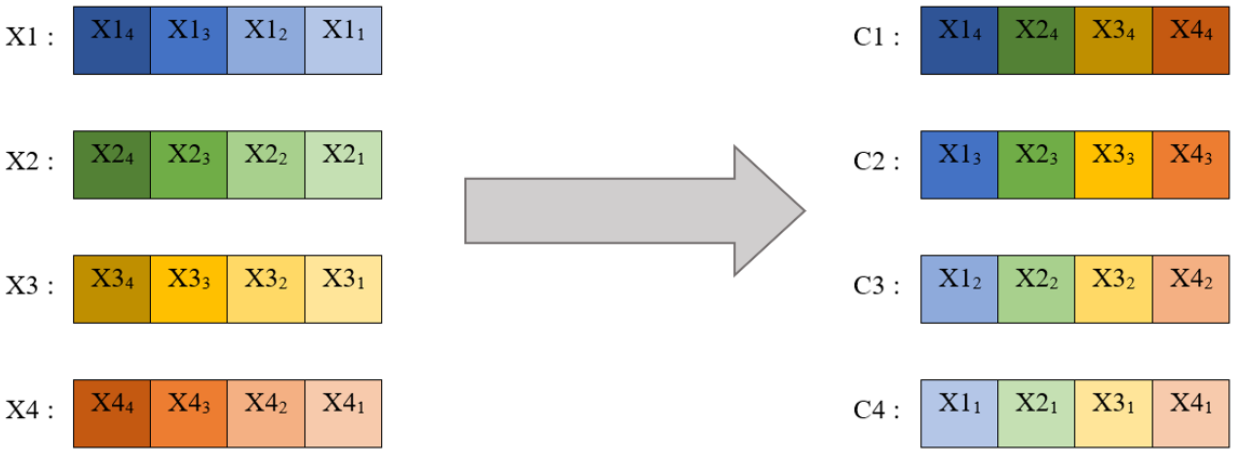
II. Sujet du TP2

Dans le TP2, il nous demande de proposer une solution de chiffrement et déchiffrement d'un code composé de 4 nombres, ou chacun est composé de 4 chiffres, à partir d'une clé tel que :

Chiffrement :

On ordonne d'une façon croissante la clé, puis on effectue la même permutation effectuée sur les chiffres de la clé, sur les chiffres des segments de code.

Ensuite la concaténation de colonne est le nombre chiffré, illustré ci-dessous :



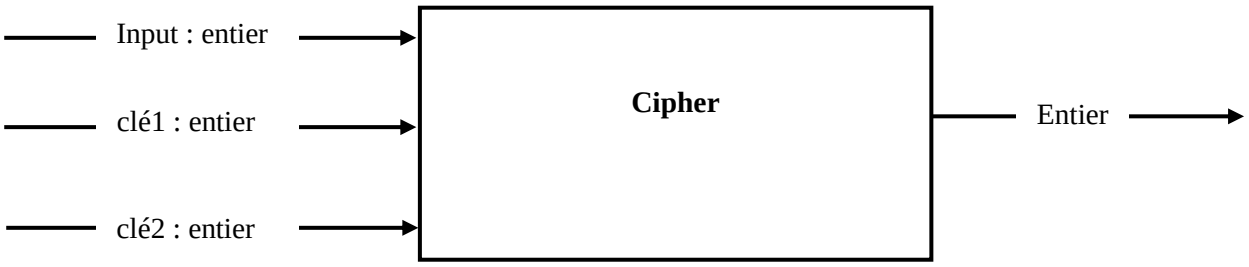
Déchiffrement :

On exécute les mêmes étapes mais dans le chemin inverse.

III. Solution Proposé

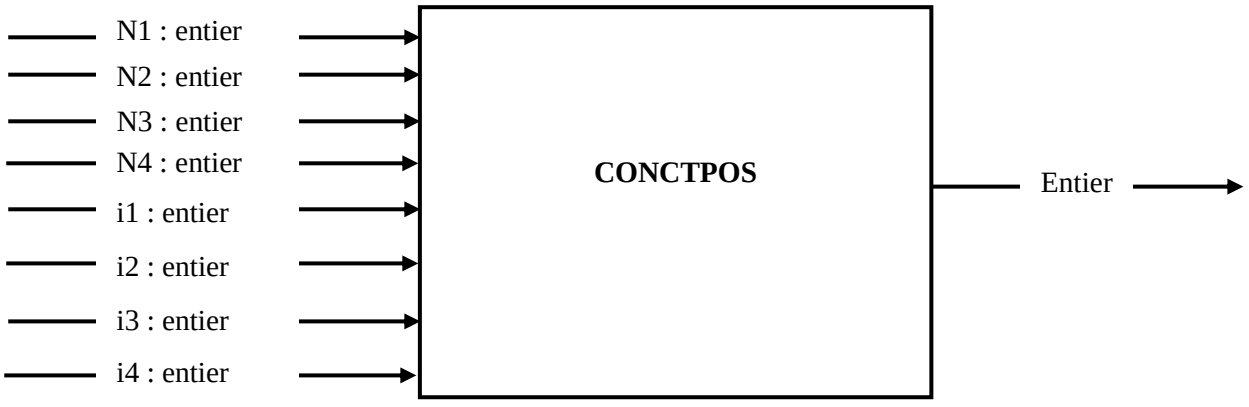
1. Découpage modulaire et sa justification

FONCTION



Rôle : Ordre les chiffre constituant ‘Input’ selon les positions d’apparition des chiffres de ‘clé2’ dans ‘clé1’ (Si les chiffre de ‘clé2’ existe dans ‘clé1’).

FONCTION



Rôle: Concatène la ‘i1’-ème position de ‘N1’ avec la ‘i2’-ème position de ‘N2’ avec la ‘i3’-ème position de ‘N3’ avec la ‘i4’-ème position de ‘N4’.

▪**Fonction** Ch_dr (input, Ch : entier) : entier //Rôle : Donne la position du chiffre ‘Ch’ le plus à droite de ‘N’ (Les chiffres sont numérotés de Droite à Gauche).

▪**Fonction** Ch_Pos (N, P : entier) : entier //Rôle : Donne le chiffre occupant la position ‘P’ de ‘N’ (Les chiffres sont numérotés de Droite à Gauche).

▪**Fonction** CON_CAT2 (A, B, p : entier) : entier //Rôle : Concatène ‘A’ avec ‘B’ à partir de la position ‘p’.

▪**Fonction** Croissant (input : entier) : entier //Rôle : Ordonne d’une façon croissant les chiffres de ‘input’.

▪**Fonction** Puiss (input, power : entier) : entier //Rôle : Donne ‘input’ à la puissance de ‘power’.

▪**Fonction** NbPos (input : entier) : entier //Rôle : Donne le nombre de positions de `input`.

Justification :

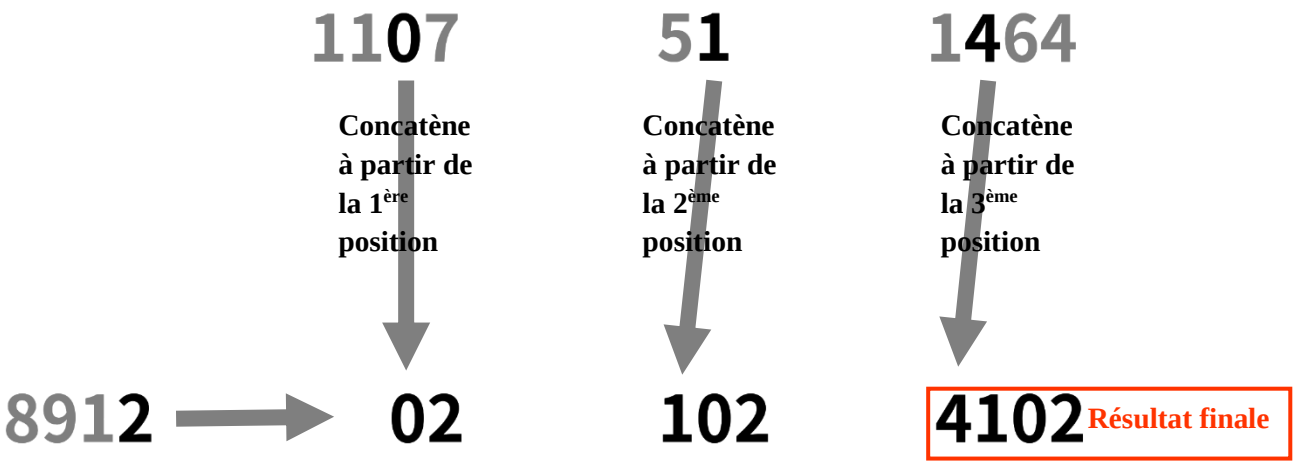
On a besoin de module *Croissant* pour ordonner la clé. Les modules *Ch_Pos* et *CON_CAT2* sont utilisé dans le module *CONCTPOS* pour spécifier quelle est la position de chiffre qu'il faut concaténer avec les autres chiffres. Le module *Puiss* est utilisé dans le module *Cipher* qui sert à effectuer les permutations sur les 4 nombres du code selon la clé, ainsi que *Ch_Pos* et *Ch_dr* pour positionner les chiffres selon leurs permutations (*NbPos* sert à trouver le nombre de chiffres du code à chiffrer).

2. Conception et réalisation

1-CONCTPOS

Analyse :

On observe l'exemple illustré ci-dessous :



Etant donné les entiers N1, N2, N3, N4 et les positions i1, i2, i3 et i4.

On concatène : le chiffre dans la position i1 de N1 à partir de la première position avec

le chiffre dans la position i2 de N2 à partir de la deuxième position avec

le chiffre dans la position i3 de N3 à partir de la troisième position avec

le chiffre dans la position i4 de N4.

Algorithme :

Fonction CONCTPOS(N1, N2, N3, N4, i1, i2, i3, i4 : entier) : entier

Var Intermédiaire : entier

Fonctions : CON_CAT2, Ch_Pos

DEBUT

Intermédiaire ← CON_CAT2(Ch_Pos(N3, i3), Ch_Pos(N4, i4), 1)

Intermédiaire ← CON_CAT2(Ch_Pos(N2, i2), Intermédiaire, 2)

CONCTPOS ← CON_CAT2(Ch_Pos(N1, i1), Intermédiaire, 3)

FIN

Jeu d'essai :

On prend :

Cas général :

N1= 3634 et **i1**= 4, **N2**= 8808 et **i2**= 3, **N3**= 6886 et **i3**= 1, **N4**= 6296 et **i4**= 4.

Le résultat est : 3866.

Cas particulier (Nombre de position de N2 < i2) :

N1= 84934 et i**1**= 5, **N2**= 329 et **i2**= 5, **N3**= 59 et **i3**= 2, **N4**= 9731 et **i4**= 1.

Le résultat est : 8**0**51.

Programme :

```
Function CONCTPOS( N1, N2, N3, N4, i1, i2, i3, i4:longint ):longint;
//-------------------------------------------------------------------------------------------------
//Role: Concatène la 'i1'-ème position de 'N1' avec la 'i2'-ème position de 'N2'
// avec la 'i3'-ème position de 'N3' avec la 'i4'-ème position de 'N4'.
// (Les zeros ne sont pas négligeable quand on concatène)
// (Si i > NbPos(N), le chiffre Ch_Pos(N,i) est un zero)
//-------------------------------------------------------------------------------------------------
Var inter:longint;                                    // inter signfie 'Intermédiaire'
begin
   inter:=CON_CAT2(Ch_Pos(N3,i3),Ch_Pos(N4,i4),1);    //Donne 'C0' dans le cas ou Ch_Pos(N4,i4) donne 0.
   inter:=CON_CAT2(Ch_Pos(N2,i2),inter,2);            //Donne 'B0D' et non pas 'BD' (dans le meme cas).
   CONCTPOS:=CON_CAT2(Ch_Pos(N1,i1),inter,3);         //Donne 'A0CD' et non pas 'ACD' (dans le meme cas).
end;
```

Résultats du jeu d'essai :

```
***** Teste de Fonction CONCTPOS *****
Donnez N1 : 3634
Donnez la position de chiffre desirable : 4
Donnez N2 : 8808
Donnez la position de chiffre desirable : 3
Donnez N3 : 6886
Donnez la position de chiffre desirable : 1
Donnez N4 : 6296
Donnez la position de chiffre desirable : 4

**********************************************
Le resultat est : 3866
```

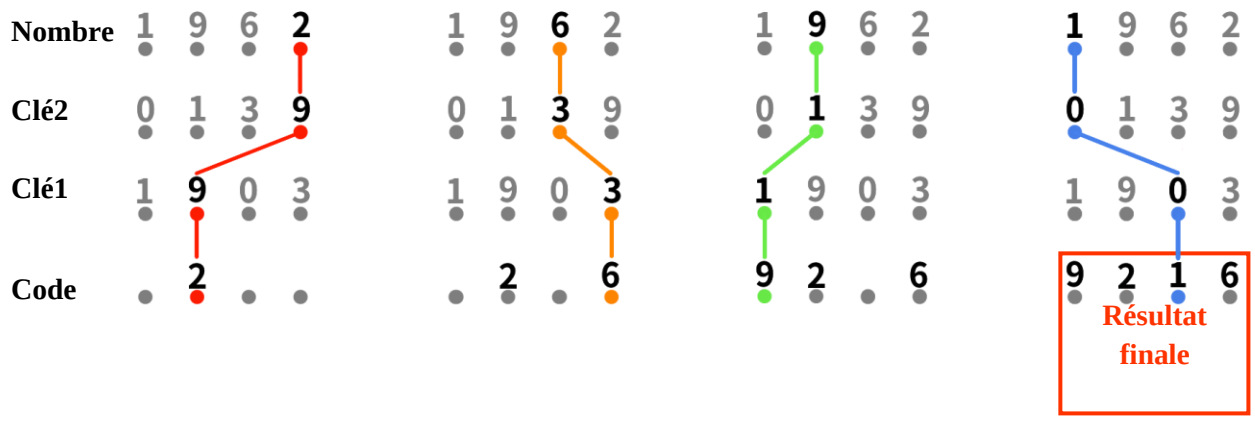
```
***** Teste de Fonction CONCTPOS *****
Donnez N1 : 84934
Donnez la position de chiffre desirable : 5
Donnez N2 : 329
Donnez la position de chiffre desirable : 5
Donnez N3 : 59
Donnez la position de chiffre desirable : 2
Donnez N4 : 9731
Donnez la position de chiffre desirable : 1

**********************************************
Le resultat est : 8051
```

2-Cipher

Analyse :

On observe l'exemple illustré ci-dessous :



Etant donné un nombre 'input' à chiffrer et deux clés ('clé1', 'clé2') :

On commence par an compteur 'i' initialisé à **1** (pour pointer vers la première position de notre nombre 'input') qui prend les valeurs entre **'1'** et **le nombre de position de 'input'** (donc *NbPos*(input)).

1. On Cherche la position d'appariation de 'i' -ème chiffre de 'clé2' (ce n'est rien que *Ch_Pos*(clé2, i)) dans 'clé1' (en utilisant *Ch_dr*).
2. On met dans 'output' (initialement vide) le 'i' -ème chiffre de 'input' (ce n'est rien que *Ch_Pos*(input, i)) dans la position qui correspond au numéro qu'on à trouver dans l'étape -1- (en utilisant *Puiss* et la multiplication, c'est $10^{position-1}$ multiplié par le 'i' -ème chiffre de 'input').
3. On répète les étapes 1 et 2 pour chaque position de 'input', on s'arrête quand 'i' est égale à *NbPos*(input) (tous les chiffres de 'input' ont été traité).

Algorithme :

Fonction Cipher(input, cle1, cle2 : entier) : entier

Variable i, Output : entier

Fonctions Ch_dr, Puiss

DEBUT

Output ← 0

Pour i **allant de** 1 **jusqu'à** NbPos(input) **Faire**

Début Pour

Output ← Puiss(10, Ch_dr(cle1, Ch_Pos(cle2, i)) -1) * (Ch_Pos(input, i)) + output;

Fin Pour

Cipher ← Output

FIN

Jeux d'essai :

On prend :

1 :

Code = 94206, **Clé1** = 41023, **Clé2** = 10342.

Le résultat est : 09462.

2 :

Code = 3291, **Clé1** = 1568, **Clé2** = 8615.

Le résultat est : 9123.

Programme :

```
Function Cipher(input,cle1,cle2:longint):longint;
//-------------------------------------------------------------------------------------------------------------------------------
//Role : Ordre les chiffre constituant 'input' selon les positions d'apparition des chiffres de 'clé2' dans 'clé1'
///(Si les chiffre de 'clé2' existe dans 'clé1')
//-------------------------------------------------------------------------------------------------------------------------------
Var i,output:longint;
Begin
 output:=0;
 For i:=1 to NbPos(input) Do output:= Puiss(10,Ch_dr(cle1,Ch_Pos(cle2,i))-1)*(Ch_Pos(input,i)) + output;
 //Le chiffrement/déchiffrement depend seulmenet de taille du code et pas la taille du clé.
 //Grace à la boucle 'For', La clé peut meme contenir un zero.
 Cipher:= output;
End;
```

Résultats du jeu d'essai :

```
***** Teste de Fonction Cipher *****
Donnez votre code : 94206
Donnez votre premiere cle : 41023
Donnez votre deuxieme cle : 10342

****************************************
Le resultat est : 9462
```

```
***** Teste de Fonction Cipher *****
Donnez votre code : 3291
Donnez votre premiere cle : 1568
Donnez votre deuxieme cle : 8615

****************************************
Le resultat est : 9123
```

3-Algorithme principal

Analyse :

On lit Cmd pour choisir entre chiffrement `C` et déchiffrement `D`

- *Pour le chiffrement :*

  On lit les nombres `A`, `B`, `C`, `D` et on les range dans un tableau d'entier appelé `Code`.

  On lit la `clé`.

  On remplit chaque cage d'un tableau `X` par l'équivalent de la fonction *Cipher* de chaque nombre du `Code` avec son équivalent de la fonction *Cipher* d'après la clé et la clé ordonné (obtenue de la fonction *Croissant*).

  On obtient le code chiffré en concaténant les positions des nombres de `X` en utilisant *CONCTPOS* (La concaténation des i -ème chiffres nous donne le (5 – i) -ème segment du code chiffré).

- *Pour le déchiffrement :*

  On lit les nombres `C1`, `C2`, `C3`, `C4` et on les range dans le tableau `Code`.

  On lit la clé.

  On remplit chaque cage d'un tableau `X` (à indice 5 – i) par la concaténation des chiffres de i -ème position de tous les nombres de `Code` (i entre 1 et 4).

  On effectue *Cipher* sur tous les cases du `Code`.

  Les cases du `Code` contient le code déchiffré.

Algorithme : // Tous qui est en gris est facultatif (n'affectent pas la logique de l'algorithme).

```
Algorithme Vigenère
Type       NB = Tableau [1..4] d'entier
Variable   Code, X : NB
           clé, i : entier
           cmd : caractère
Fonctions CHR, Cipher, CONCTPOS, Croissant
Procédure clrscr

DEBUT
```

```
Répéter
  Ecrire('Ecrire "C" pour chiffrement et "D" pour déchiffrement : ')
  Lire(cmd)
  clrscr
Jusqu'à ( cmd='c' ) ou ( cmd='C' ) ou ( cmd='d' ) ou ( cmd='D' )
Si (cmd='c') ou (cmd='C') alors
  Début Si
  Pour i allant de 1 jusqu'à 4 faire
    Début Pour
    Ecrire(CHR(64+i),'= ')
    Lire(Code[i])
    Fin Pour
  Saut de ligne
  Ecrire('clé= ')
  Lire(clé)
  Saut de ligne
  Pour i allant de 1 jusqu'à 4 Faire X[i] ← Cipher(Code[i], cle, Croissant(cle))
  Ecrire('Le code après le chiffrement est : ')
  Pour i allant de 4 jusqu'à 1 Faire Ecrire(CONCTPOS( X[1], X[2], X[3], X[4], i, i, i, i),' ')
  Fin Si
Sinon
  Début Sinon
  Pour i allant de 1 jusqu'à 4 faire
    Début Pour
    Ecrire( 'C', i, '= ')
    Lire(Code[i])
    Fin Pour
  Saut de ligne
```

```
    Ecrire('clé= ')
    Lire(clé)
    Saut de ligne
    Pour i allant de 1 jusqu'à 4 Faire X[5 - i]←CONCTPOS(Code[1],Code[2],Code[3],Code[4],i,i,i,i)
    Ecrire('Le code après le déchiffrement est : ')
    Pour i allant de 1 jusqu'à 4 Faire Ecrire(CHR(64+i),'= ', Cipher( X[i], Croissant(cle), cle),' ')
    Fin Sinon
FIN
```

Jeu d'essai :

Chiffrement :

On prend :

1 :

A = 8518, **B** = 6132, **C** = 8549, **D** = 2409, **clé** = 4520.

Le résultat est : **C1** = 1340, **C2** = 8299, **C3** = 5154, **C4** = 8682.

2 :

A = 4158, **B** = 5785, **C** = 2661, **D** = 7122, **clé** = 4385.

Le résultat est : **C1** = 1761, **C2** = 4527, **C3** = 8512, **C4** = 5862.

Déchiffrement :

On prend :

1 :

C1 = 1340, **C2** = 8299, **C3** = 5154, **C4** = 8682, **clé** = 4520.

Le résultat est : **A** = 8518, **B** = 6132, **C** = 8549, **D** = 2409.

2 :

C1 = 1761, **C2** = 4527, **C3** = 8512, **C4** = 5862, **clé** = 5862.

Le résultat est : **A** = 4158, **B** = 5785, **C** = 2661, **D** = 7122.

Observation :

On prend :

1 :

A = 3634, **B** = 8858, **C** = 6886, **D** = 6296, **clé** = 1302.

Le résultat est : **C1** =6882, **C2** = 4866, **C3** = 3866, **C4** = 3589.

2 :

A = 3634, **B** = 8858, **C** = 6886, **D** = 6296, **clé** = 4927.

Le résultat est : **C1** =6882, **C2** = 4866, **C3** = 3866, **C4** = 3589. (Même résultat que 1 :).

Programme :

```
Program Vigenere_last;
Uses crt, Socrates, Platon;                                    //Socrates et Platon contiennent Cipher, CONCTPOS et Croissant
Type NB= Array [1..4] of integer;
Var Code,X:NB;   cle:integer;
    cmd:char;        i:byte;
Begin
  Repeat
  Write('Ecrire "C" pour chiffrement et "D" pour dechiffrement : ');
  Read(cmd);
  clrscr;
  Until (cmd='c') or (cmd='C') or (cmd='d') or (cmd='D');       //Cette boucle est pour enforcer le choix entre C ou D.
  If (cmd='c') or (cmd='C') then
  begin
   For i:=1 to 4 Do
    Begin
     Write(CHR(64+i),'= ');                                    //CHR(64+i) donne 'A' dans la 1ere iteration, 'B' 2eme, 'C' 3eme et 'D'  4eme.
     Read(Code[i]);
    End;
   writeln;
   write('cle= ');
   Readln(cle);
   writeln;
   For i:=1 to 4 Do X[i]:=Cipher(Code[i],cle,Croissant(cle));
   Write('Le code apres le chiffrement est : ');
   For i:=4 downto 1 Do Write(CONCTPOS(X[1],X[2],X[3],X[4],i,i,i,i),' ');
   End
  Else
  begin
   For i:=1 to 4 Do
    Begin
     Write('C',i,'= ');
     Read(Code[i]);
    End;
   writeln;
   write('cle= ');
   Readln(cle);
   writeln;
   For i:=1 to 4 Do X[5 - i]:=CONCTPOS(Code[1],Code[2],Code[3],Code[4],i,i,i,i);
   Writeln('Le code apres le dechiffrement est :');
   For i:=1 to 4 Do Writeln(CHR(64+i),'= ', Cipher(X[i],Croissant(cle),cle),' ');          //CHR(64+i) donne 'A' dans la 1ere iteration, 'B' 2eme, 'C' 3eme et 'D'  4eme.
  end;
End.
```

Résultats du jeu d'essai :

Chiffrement :

```
A= 8518
B= 6132
C= 8549
D= 2409

cle= 4520

Le code apres le chiffrement est : 1340 8299 5154 8682
```

```
A= 4158
B= 5785
C= 2661
D= 7122

cle= 4385

Le code apres le chiffrement est : 1761 4527 8512 5862
```

Déchiffrement :

```
C1= 1340
C2= 8299
C3= 5154
C4= 8682

cle= 4520

Le code apres le dechiffrement est :
A= 8518
B= 6132
C= 8549
D= 2409
```

```
C1= 1761
C2= 4527
C3= 8512
C4= 5862

cle= 4385

Le code apres le dechiffrement est :
A= 4158
B= 5785
C= 2661
D= 7122
```

Observation :

```
A= 3634
B= 8858
C= 6886
D= 6296

cle= 1302

Le code apres le chiffrement est : 6882 4866 3866 3589
```

```
A= 3634
B= 8858
C= 6886
D= 6296

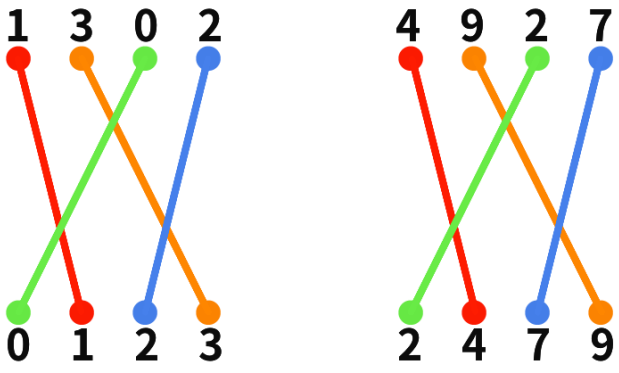
cle= 4927

Le code apres le chiffrement est : 6882 4866 3866 3589
```

Elaboration :

Ce chiffrement peut donner le même résultat pour des clés différentes parce qu'il est positionnel (basé sur les positions des chiffres) :

On effet, les clés 1302 et 4927 sont identiques. Quand on observe les permutations affectées sur leurs chiffres pour les ordonner, on trouve la même paterne :



À priori, il y avait 3024 clés possible pour chiffrer un nombre de 4 positions (3024=9×8×7×6), mais en réalité c'est 24 paternes possibles (24=4×3×2×1), une de ces paternes n'est pas un chiffrement (clé=1234) tandis qu'une autre est une inversion mirrore simple (clé=4321) (Sans rendre compte des permutations d'un seul chiffre), donc un peu proche de 22 clés effectives.

IV. Conclusion

Pour conclure, notre solution est loin d'être parfaite, mais on a appris beaucoup de choses de ce TP. D'une face, le travail en binôme demande une très bonne organisation, en fait, c'est sa pierre angulaire. De l'autre face, les objets structurés facilitent trop la tâche par rapport au objets simples. En plus, la réalisation des modules générales à l'avance est très utile car elle permet de ne pas perdre du temps.

Le chiffre de Vigenère étant très simplifiée (pour des raisons pédagogiques), il a devenu très susceptible d'être déchiffré par une simple attaque par force brute ; Ironiquement, un chiffre facilement déchiffrable.

Enfin, ce TP reste une expérience qui nous a fait connaître nos lacunes pour les régler le plus vite possible.